Where 4-Bit Quantization Breaks Small Tool-Calling Agents, and Which Repairs Fix It

CMPE 295A · Workbook 1 · October 9, 2026

Ekant Kapgate **Vritika Malhotra** **Yashashav Devalapalli Kamalraj**

Advisor: Dr. Bertin Cordova-Diba

Presented by Yashashav Devalapalli Kamalraj for the team

Small agents ship at 4 bits, and the damage hides

All

8.0 → 2.5 GB

Qwen3-4B weights, 16-bit vs 4-bit Q4_K_M
Taheri et al. [19]

19.5% → 38.3%

Calls to tools that don't exist, 16-bit → 4-bit; task score shows no significant change
τ^2-bench, Gemma-4-31B, Jang et al. [21]

62.3% → 16.8%

Qwen3-0.6B on BFCL single-turn at Q4_K_M
Taheri et al. [19]

Aggregate scores hide where 4-bit breaks small agents.

Three repairs, never compared

Yashashav Vritika

Repair	What it changes	Judged by	Gap
R1 Restrict output (grammar)	Decoder masks invalid tokens	Tool-call validity, speed	Can’t fix valid-but-wrong calls
R2 Add bits	Precision of some layers or steps	Perplexity, continuation scoring	Step vs layer vs uniform untested on tool calling
R3 Retrain (QAD)	Weights, same 4-bit format	Aggregate scores	No train-longer control

RQ1 Where does 4-bit damage concentrate (step type, depth)?

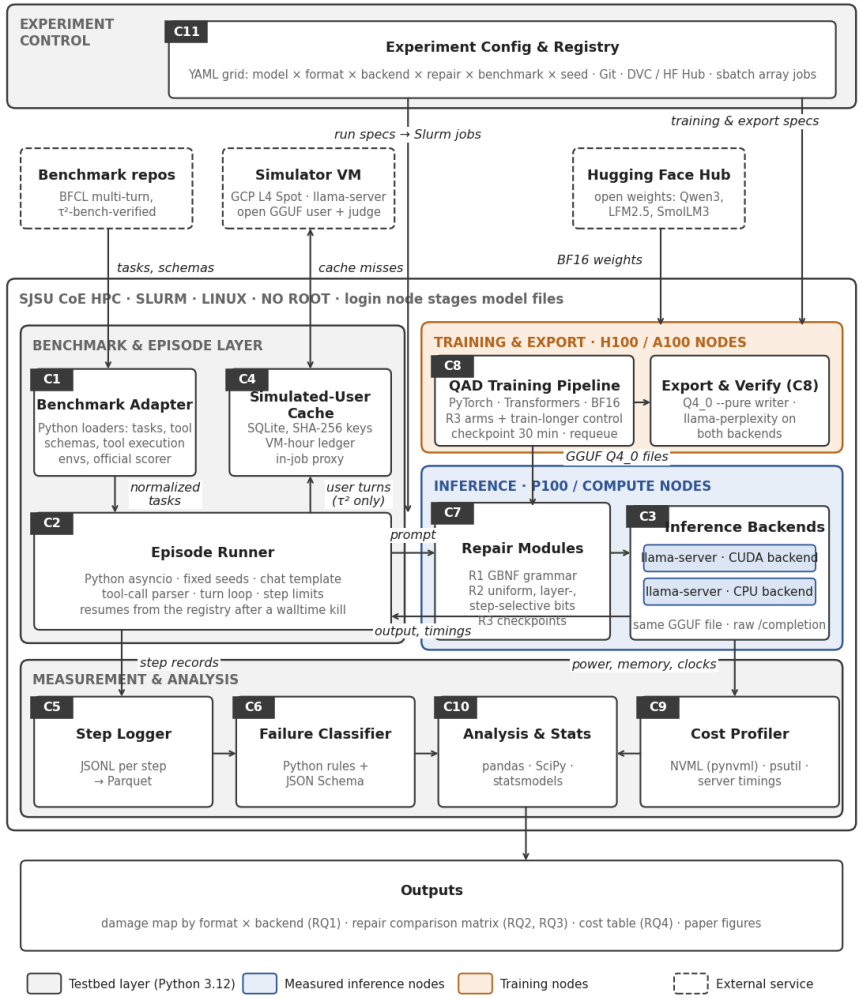
RQ2 How do the three repairs compare, each with its control?

RQ3 Do repairs fix the failures or only the score?

RQ4 What does each repair cost to deploy?

Testbed architecture

Ekant Vritika Yashashav



- Pipeline: config → episodes → step logs → failure labels → statistics
- Pluggable inference: llama.cpp CUDA and CPU backends on the same GGUF file; repairs are hooks
- Every agent step logged and labeled with one of 10 failure types

Ekant — runner, backends, logger, R1, cost

Vritika — benchmarks, simulator cache, classifier, stats

Yashashav — R2, QAD pipeline, conversion, registry

Study design

Vritika Yashashav

Axis	Values
Models	Qwen3 0.6B / 1.7B / 4B · LFM2.5 1.2B / 2.6B · SmolLM3-3B
Formats	BF16 reference · Q4_0 · Q4_K_M · IQ4_XS on CUDA; Q4_0 also on CPU
Repairs (control)	R1 grammar (no grammar; grammar at 16-bit) · R2 Q5_0/Q6_K/Q8_0, 5.5-bit layer-selective, two-copy step routing (Q4_0) · R3 QAD-KL, QAD-KL+CE (train-longer at full precision)
Benchmarks	BFCL multi-turn (300) · τ^2-bench-verified (278) · BFCL single-turn (1,000)
Metrics	Task success, pass^k, failures per type, amplification ratio, memory / TTFT / tok/s / energy

30 functional + 11 non-functional requirements, each with an acceptance test · analysis pre-registered on OSF at M2 (Nov 6)

Progress, Sprints 1–3 (Aug 31 – Oct 9)

Ekant | Vritika | Yashashav

Sprint	Ekant	Vritika	Yashashav
S1 Aug 31–Sep 13	Reading list: runtimes, GGUF, constrained decoding (T2)	Reading list: benchmarks, evaluation (T3) Team-formation slides (A2) Abstract rework and class talk, with Yashashav (A1, T1)	Abstract rework and class talk, with Vritika; advisor-approved Sep 3 (A1, T1) Reading list: mixed precision, QAD (T4) Problem statement + plan draft (T5)
S2 Sep 14–27	GitHub repo + Projects board (T6) WB1 Ch5, Ch8 (T10) M1–M8 plan + Gantt, with Yashashav (A6) *Laptop llama.cpp CPU build (T11, in progress)*	WB1 Ch3 requirements (T8) Refined abstract (A7) *bfcl-eval + τ^2-bench setup (T12, in progress)*	RQ1–RQ4 problem statement (A5) M1–M8 plan + Gantt, with Ekant (A6, T14) WB1 Ch2, Ch4, Ch9 (T9) *Repo layout + Dockerfile (T13, in progress)*
S3 Sep 28–Oct 9	Compute sizing + HPC request, with Yashashav (T15, A9)	Methodology, Ch3 + Ch7 (A8) Analysis policy reconciled (T16) Advisor review arranged (T18)	References re-verified (T17) HPC request, with Ekant (A9) **First NVIDIA pilot runs, Oct 8–9**

Team Ch1 literature review, 117 references (T7) · WB1 submitted: 83 pp, 40 tables, 6 figures · 7 meetings logged

Open S2/S4 logs (Ekant, Vritika, due Oct 9, T19) · GCP L4 quota (Yashashav, T23) · advisor meeting, week of Oct 12 (Yashashav, T22) · HPC accounts pending since Sep 30

Source: team meeting log, github.com/VritikaMalhotra/CMPE_295A/meetings (action-item IDs in brackets; italic = in progress)

First NVIDIA pilot runs (Colab A100, Oct 8)

Ekant (owner) | Yashashav (ran)

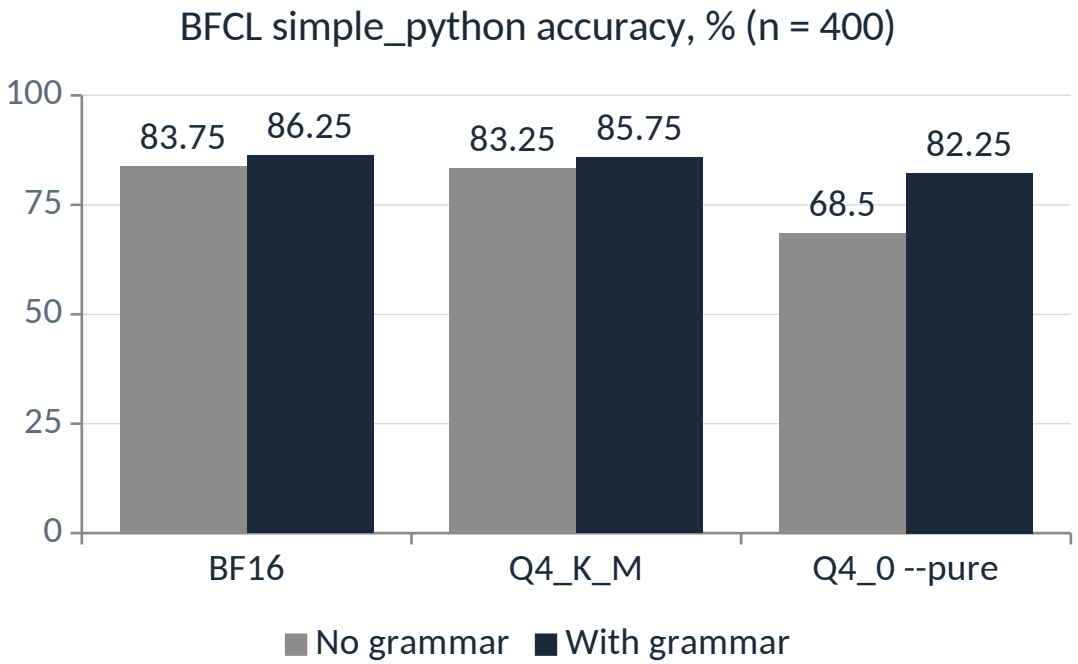
Format	Bits / weight	Decode tok/s	Peak GPU MiB	mJ / token
BF16	16.06	291.5	3096	349
Q4_0 --pure	4.57	379.3	2262	134
Q4_K_M	5.15	342.2	2316	185

Q4_K_M keeps 28.5% of weights at Q6_K, so it measures 5.15 bits/weight here, not the ~4.9 we planned with. We log measured size.

Q4_K_M ≈ BF16 (83.00 vs 83.75%, p = 0.66): published collapse not reproduced here

Q4_0 --pure −15.75 points: mostly skipped calls (60 vs 11 tasks)

Grammar: format failures → 0, Q4_0 up to 82.25%; wrong arguments remain



Format failures, without → with grammar
BF16 14 → 0 · Q4_K_M 14 → 0 · Q4_0 66 → 0
Both bars use the same endpoint (raw /completion); the bfcl run on the left gave 83.75 / 83.00 / 68.00.

Pilot / development data, excluded from the study. Run by Yashashav (backup owner C3/R1) on a Colab A100, 2026-10-08 18:54-19:50 PT, while CoE HPC access is pending (DEP-4 fallback). Qwen3-0.6B, llama.cpp v0.6.0, bfcl-eval 2026.3.23, greedy, one seed.

What could stop us

Yashashav

DEP-4 | Pending | Owner: Ekant, Yashashav

HPC accounts pending since Sep 30

Fallback: Colab A100 in use now (slide 7); laptops run the walking skeleton

DEP-8 | High for 0.6B | Owner: Vritika

Qwen3-0.6B solves too few multi-turn tasks

Fallback: add the BFCL single-turn set; a model under 120 solvable tasks gets single-turn analysis only

DEP-3 | Medium | Owner: Vritika, Yashashav

No GPU to host τ^2-bench's simulated user

Fallback: GCP L4 Spot, about $130; cluster A100; BFCL alone still covers RQ1–RQ3

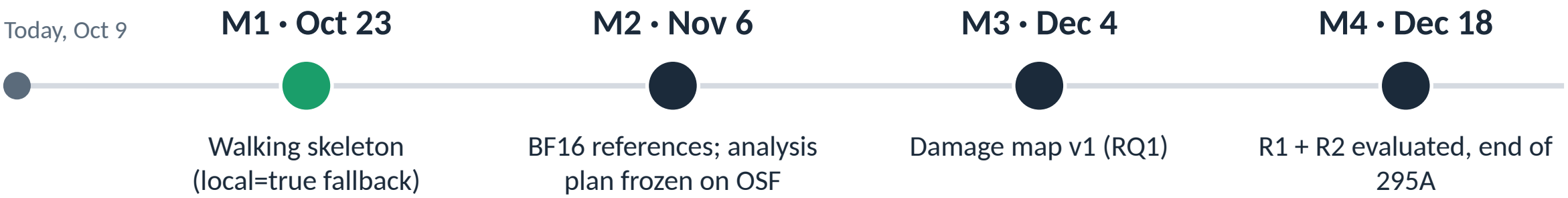
DEP-9 | Medium | Owner: All

Someone publishes the comparison first

Fallback: OSF timestamp at M2; our controls and measured costs stay new

Next three weeks

Ekant · Vritika · Yashashav



Before M1, by member

Ekant — Rerun S1/S3/S6 · C2 runner, C3 backends, C5 logger · R1, C9 profiler, CI

Vritika — S4 multi-turn + C1 loaders · S5 simulator · C4 cache, C6 rules v0, C10 toy stats

Yashashav — S8 export · C11 registry + GCP quota · R2 Q8_0 + S9 switch · Dockerfile

Deliverables

Backup

ID	Deliverable	Target	Owner
DL-1	Open-source testbed, C1–C11	v0.1 at M4; v1.0 + DOI at M8	Ekant, Yashashav
DL-2	Failure taxonomy, classifier, κ report	v0 at M2; v1 at M3	Vritika
DL-3	Pre-registered analysis plan (OSF)	M2	Vritika
DL-4	Damage map (RQ1)	v1 at M3; final at M8	Vritika
DL-5	Repair comparison matrix (RQ2, RQ3)	R1, R2 at M4; R3 at M5; costs at M6	Ekant, Yashashav, Vritika
DL-6	QAD pipeline with exact Q4_0 export	M5	Yashashav
DL-7	Cost measurements (RQ4)	M6	Ekant
DL-8	Step-log dataset (Hugging Face, Zenodo DOI)	v1 after M7; final at M8	Vritika
DL-9	Workshop paper (EuroMLSys 2027)	M7	All

Also AD-1 retrained checkpoints on Hugging Face (Yashashav) · AD-2 one-command reproducibility package (Yashashav, Ekant) · AD-4 practitioner guide, “which repair when” (Yashashav, lead)

Metrics and hypotheses

Backup

Key metrics

- Task success, pass^k
- Step failures by type
- Amplification ratio vs 16-bit
- Flip rate
- Gap closed by a repair
- Memory, TTFT, tok/s, energy

H1 (RQ1) 4-bit raises per-step failures above 16-bit; the excess concentrates in tool-call steps and at depth

H2 (RQ2) Each repair cuts excess failures vs its control; R1 fixes format failures, not semantic ones; QAD beats train-longer

H3 (RQ3) Repairs close more of the score gap than of the failure gap

H4 (RQ4) R1 adds no weight memory and at most 5% to median per-step decode latency on CUDA

State of the art in industry

Backup

Vendor	Release	Technique	Target
Meta	Llama 3.2 1B/3B, Oct 2024	QAT with LoRA; SpinQuant	4-bit weights, 8-bit activations
Google	Gemma 3 QAT, Apr 2025	QAT toward the unquantized model	Q4_0 for llama.cpp, Ollama, MLX
Apple	On-device model, 2025	2-bit QAT + LoRA recovery adapters	Apple silicon
NVIDIA	NVFP4 QAD, 2026	KL distillation from BF16 teacher	NVFP4 GPUs
Google	Gemma 4 QAT, Jun 2026	QAT; Q4_0 plus a mobile format	E2B at about 1 GB
Multiverse	Quantization-aware healing, 2026	Distillation into a 4-bit model	MXFP4
Liquid AI	LFM2.5 QAD, Aug 2026	QAD for 230M to 2.6B models	GGUF Q4_0

Structured output ships in OpenAI, vLLM/SGLang (XGrammar) and llama.cpp (GBNF), and every one of these is judged on aggregate scores.
No vendor reports per-failure-type agent results for its quantized builds or compares the three repairs.

Ownership and backups

Backup

Member	Owns	Key deliverables	Backup for
Ekant	C2 runner, C3 backends, C5 logger, R1 grammar, C9 cost profiler	Testbed core on CUDA + CPU; cluster builds, Slurm runner; v0.1 + CI; R1 grammars; RQ4 cost matrix; Expo demo	C1, C4, R2, C8
Vritika	C1 benchmark adapter, C4 simulator cache, C6 failure classifier, C10 analysis + stats	Inclusion set; simulator + cache; codebook + labeled sample; OSF pre-registration; damage map (RQ1); RQ3 analysis	C2, C5, C9, C11
Yashashav	R2 added bits, R3 + C8 QAD pipeline, conversion + 4-bit checks, C11 config + registry	R2 variants; QAD checkpoints on the Hub; verified GGUF exports; lockfile, Dockerfile, Singularity; v1.0 + DOI	C3, C6, R1, C10

Rule: the owner or the backup runs every spike. The Oct 8 pilot ran under Yashashav as backup for C3 and R1.

Full schedule

Backup

